# 1. MongoDB pour les données du jeu et Redis pour le classement

- **Date** : 2026-13-10
- **Statut** : Proposée

## Contexte

Notre projet consiste à développer un jeu en ligne dans lequel les joueurs peuvent créer des personnages, gérer leurs inventaires et obtenir des points en participant à des activités.

Chaque personnage possède ses propres caractéristiques (nom, niveau, expérience et classe). Les inventaires contiennent des objets variés (armes, armures, potions, accessoires), avec une quantité et un état d'équipement. Ces données peuvent évoluer au cours du jeu.

Le classement doit afficher les joueurs selon leur nombre total de points. Il est donc consulté régulièrement et doit pouvoir être mis à jour rapidement.

Nous utilisons Node.js pour développer l'application et Docker pour lancer les différents services.

## Options envisagées

1. **PostgreSQL pour toutes les données**. Pour : les relations entre joueurs, personnages, inventaires et objets sont bien structurées ; les contraintes et les transactions permettent de garantir la cohérence des données. Contre : les caractéristiques des personnages et des objets peuvent évoluer, ce qui peut nécessiter des modifications du schéma.
2. **MongoDB pour les données du jeu**. Pour : les documents JSON permettent de stocker facilement des données variables, notamment les caractéristiques des personnages et les objets présents dans les inventaires. Les données liées peuvent être intégrées dans un même document. Contre : les relations entre les collections ne sont pas automatiquement garanties ; l'application doit vérifier la cohérence des références.
3. **Redis pour stocker les données du jeu et le classement**. Pour : les lectures et les mises à jour sont très rapides ; Redis propose les Sorted Sets, adaptés au classement des joueurs. Contre : Redis est moins adapté au stockage principal de données structurées et à la gestion de relations complexes entre joueurs, personnages, inventaires et objets.

## Décision

Nous retenons une solution combinant **MongoDB et Redis**.

MongoDB stockera les données principales du jeu : joueurs, personnages, inventaires, objets et scores. Son format documentaire permet de faire évoluer facilement les informations des personnages et des objets.

Redis sera utilisé uniquement pour le classement. Les Sorted Sets permettront de classer les joueurs selon leur total de points et de récupérer rapidement les meilleures positions.

Cette séparation permet d'utiliser chaque base de données pour le type de données auquel elle est le mieux adaptée.

# Conséquences

Plus facile : ajouter de nouvelles caractéristiques aux personnages et aux objets sans modifier une structure relationnelle complexe. Le classement peut également être consulté et actualisé rapidement grâce à Redis.

Plus difficile : garantir la cohérence des références entre les collections MongoDB. Il faut aussi maintenir la synchronisation entre les scores enregistrés dans MongoDB et les points stockés dans Redis. L'application Node.js devra donc gérer ces vérifications.

À revoir si : le volume de données devient très important, si les relations entre les données nécessitent davantage de contraintes, ou si la synchronisation entre MongoDB et Redis devient trop difficile à maintenir.